

DEVELOPER PORTFOLIO

안도형

Backend & Infrastructure Engineer —

EMAIL dksehgud1@naver.com

GITHUB github.com/dksehgud

PHONE 010-2613-3584

AWARDS 삼성전자 SW 역량테스트 B형 보유

"팀의 탱커이자 서포터 — 견고한 백엔드 구조와 안정적인 배포 인프라로 서비스를 받치는 개발자"

눈에 잘 보이지 않는 아키텍처와 배포 인프라가 팀의 개발 효율과 서비스 안정성을 받치는 기반이라고 생각합니다. 배포 흐름, 비동기 통신, 데이터 정합성에서 문제가 생기면 로그와 구조를 함께 확인하고, 같은 문제가 반복되지 않도록 개선하는 개발자가 되고 싶습니다.

Back-End

Spring Boot 3 · Spring Security · FastAPI · JPA / MyBatis

Infra · DevOps

Docker · Nginx · GitLab CI/CD · AWS EKS · ArgoCD · Kafka

Data · Client

MySQL · Redis / Redisson · ShedLock · TypeScript · Kotlin WebView

NAME

안도형 (Ahn Do-hyung)

ROLE FOCUS

Backend / Infra

CORE STACK

Java · Spring Boot · Python/FastAPI
· TypeScript · AWS

CERTIFICATION

삼성전자 SW 역량테스트 B형

AFFILIATION

가천대학교 졸업
SSAFY 14기 수료 예정

EDUCATION

- 2025.07 ~ 2026.06
삼성 청년 SW 아카데미 (SSAFY)
Java/Python 기반 백엔드 및 클라우드 아키텍처 과정 수료 예정
- 2025.02
가천대학교 컴퓨터공학과 학사 졸업

AWARDS & CERTIFICATIONS

- 2026.04
삼성전자 SW 역량테스트 B형 취득
C++/Java 기반 고성능 자료구조 구현 및 최적화 역량 검증
- 2026.05
우수상 — ReBloom 프로젝트
ReBloom (청소년 정신건강 사후 추적관리 서비스)
- 2026.04
우수상 — 삼성전자 DX부문 기업 협력 프로젝트
S-Catch (AI 적응형 글로벌 웹 크롤링 서비스)
- 2026.02
우수상 — SSAFY 자율 프로젝트
RE:FIT (AI 가상 피팅 중고 공유 플랫폼 - 인프라 전담)

Back-End

API & Business Logic

Spring Boot 3.x

FastAPI

Spring Security

JPA / MyBatis

Kafka

Spring Boot로 REST API, 인증/인가, 스케줄러, 비동기 처리, 예외 처리 구조를 구현했습니다. FastAPI로 S-Catch의 크롤링 작업 생성, 상태 조회, 결과 비교 API를 구현했습니다.

Infra & DevOps

Cloud & Pipeline

Docker

Nginx

AWS EKS

ArgoCD

GitLab CI/CD

Docker, Nginx, GitLab CI/CD로 RE:FIT 배포 자동화를 구현했습니다. AWS EKS와 ArgoCD로 ReBloom 7개 MSA 서비스를 배포하고 운영 지표를 확인했습니다.

Data & Client

Storage / UI / Mobile

MySQL

Redis

Redisson

ShedLock

TypeScript

Kotlin WebView

Redis, Redisson, ShedLock으로 대기열, 동시성, 스케줄러 중복 실행 문제를 다뤘습니다. TypeScript로 S-Catch 진행률 UI를 구현했고, Kotlin WebView로 ReBloom 모바일 브릿지를 구현했습니다.

PROJECTS LIST

총 4개의 프로젝트 상세 내역

01	ReBloom 우수상	상담 이후 상태를 생체·대화 데이터로 추적하는 서비스	Spring Boot · EKS · Kafka · WebView
02	S-Catch 우수상	91개국 삼성닷컴 상품 정보를 수집·비교하는 크롤링 서비스	FastAPI · TypeScript · Playwright · SSE
03	RE:FIT 우수상	중고 의류 거래 서비스의 배포와 CI/CD를 구축한 프로젝트	Spring Boot · Docker · Blue/Green
04	Tget	Redis 대기열로 예매 진입 순서를 제어한 공연 예매 서비스	Spring Boot 3 · Redis · MyBatis · Claude

인프라 · 백엔드 코어 · 하이브리드 앱 브릿지

ReBloom

청소년의 상담 이후 상태를 생체 데이터와 대화 패턴으로 수집하고 추적하는 서비스.

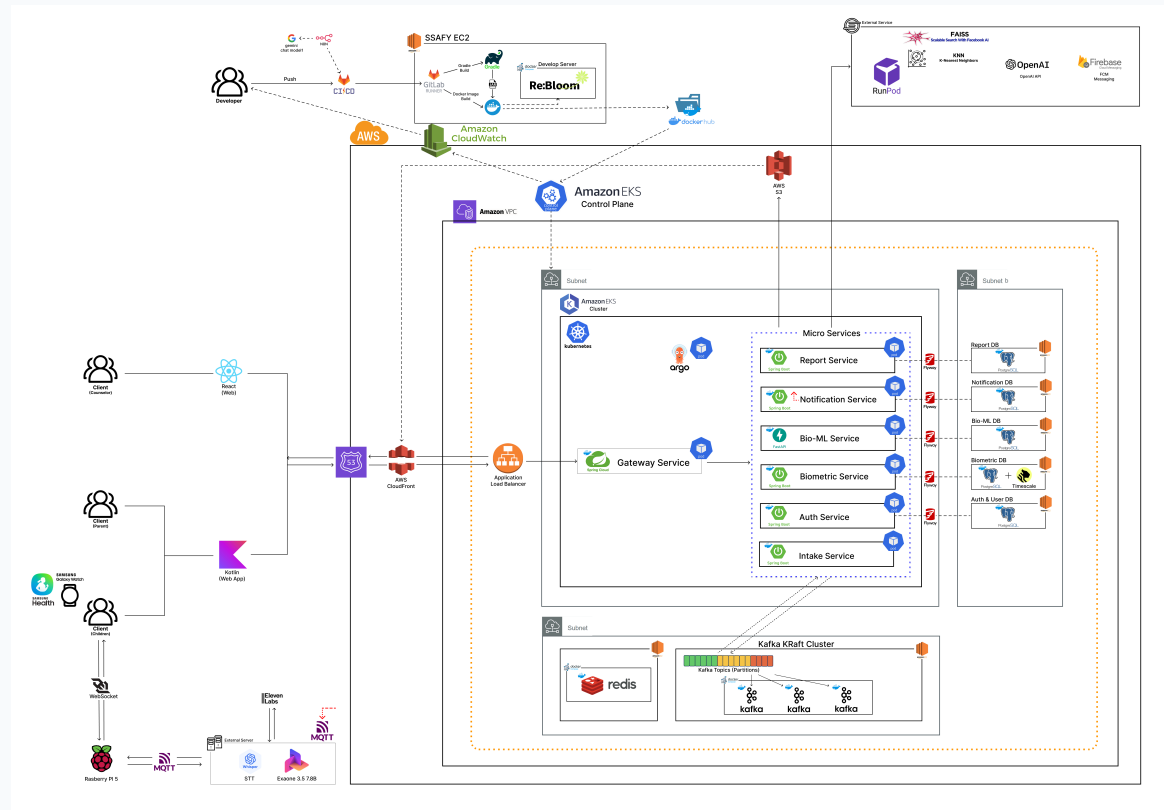
7개 MSA를 EKS에 배포하고 Kafka 이벤트 처리, Redis/ShedLock 스케줄러, Kotlin WebView Bridge를 구현했습니다.

 **기간:** 2026.04 ~ 2026.05 (6주)

 **팀 구성:** 6명 (BE 2, FE 2, AI 2)

 **GitHub:** github.com/dksehgud/ReBloom

01. ReBloom 시스템 아키텍처



EKS 기반 7개 MSA · [Kafka](#) 비동기 이벤트 스트리밍 · [ShedLock + Redis](#) 분산 스케줄링 · ArgoCD GitOps · Kotlin WebView 하이브리드 클라이언트

AWS EKS

Kafka

ArgoCD

MySQL

Redis

FastAPI

I ① 문제 정의

여러 서비스가 EKS에서 동시에 실행되면서 스케줄러 중복 실행, 비동기 이벤트 추적, 모바일 WebView의 오프라인 저장과 센서 접근 처리가 필요했습니다.

I ② 문제 원인

- Spring `@Scheduled`는 Pod마다 실행되어 같은 배치가 여러 번 돌 수 있음.
- Kafka 비동기 처리 구간에서 요청 흐름을 추적할 공통 ID가 필요함.
- 모바일 앱에서 웹 화면을 띄우는 과정에서 팝업, 뒤로가기, 로컬 저장 흐름을 별도로 보완해야 했습니다.

I ③ 해결 과정

- **ShedLock + Redis**로 다중 Pod 환경의 스케줄러 중복 실행을 막는 구조를 구현했습니다.
- **Kafka** 메시지에 `correlationId`를 넣고 MDC에 연결해 로그 추적 흐름을 만들었습니다.
- Android WebView 설정과 Native Bridge를 보완해 앱 안에서 팝업, 뒤로가기, 로컬 저장 흐름이 자연스럽게 동작하도록 구현했습니다.

I ④ 결과 (AWS · GitLab 실측 기준)

7/7 EKS 7개 MSA Pod Running
(재시작 0회)

7,558건 ALB 누적 요청 · 평균
TargetResponseTime
41ms

318K건 CloudWatch 로그 이벤트
(1주 운영 추적)

Synced ArgoCD GitOps ·
Image Updater 7개 이
미지

FastAPI · TypeScript · Playwright · SSE

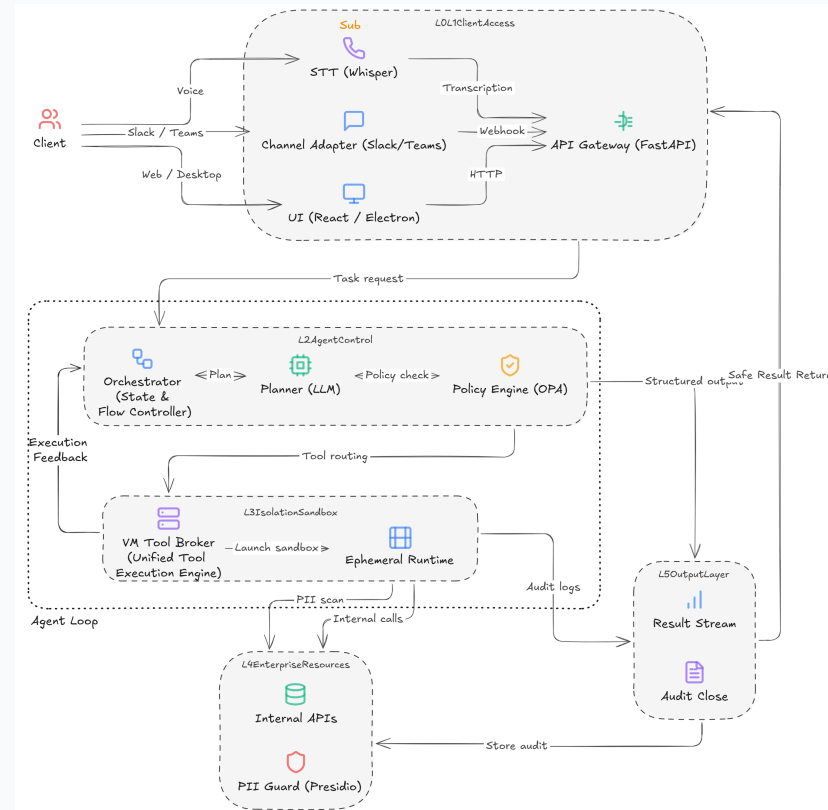
S-Catch

삼성닷컴 91개국 상품 페이지에서 TV 스펙 정보를 수집하고 기준 데이터와 비교하는 서비스.
국가별 사이트 구조를 3유형으로 나누고 FastAPI, TypeScript, Playwright, SSE로 수집 진행과 결과 비교를 구현했습니다.

📅 기간: 2026.02 ~ 2026.04 (6주)

👥 팀 구성: 6명

🔗 GitHub: github.com/dksehgud/s-catch



FastAPI + TypeScript 작업 관리 UI · Playwright 수집 엔진 · SSE 실시간 진행률 · 기준 데이터
Diff 비교 · 91개국 구조 테스트

FastAPI

Playwright

TypeScript

SSE

Diff API

Job State

I ① 문제 정의

국가마다 삼성닷컴 상품 목록과 상세 페이지 구조가 달라 수집 코드가 쉽게 깨졌고, 오래 걸리는 크롤링 작업의 진행 상태를 화면에서 확인하기 어려웠습니다.

I ② 문제 원인

- 91개국 사이트가 한국형·미국형·기타 지역형으로 나뉘어 DOM/API 구조가 달랐음.
- 수집 시간이 길어 사용자가 현재 단계와 실패 여부를 확인하기 어려웠음.
- SSE 연결이 끊기거나 서버가 재시작되면 작업 상태가 어긋날 수 있었음.

I ③ 해결 과정

- **Playwright** 기반 수집 흐름을 공통화하고, 국가별 구조 차이는 3유형으로 나누어 처리했습니다.
- 작업 상태를 저장하고 SSE로 진행률을 보내 재접속 시에도 화면 상태를 복구했습니다.
- 기준 데이터와 수집 결과를 비교해 matched/mismatched/missing/unexpected로 분류했습니다.

I ④ 결과 (구현 기준)

6개 API 작업 생성·상태 조회
·SSE·결과 비교 API

4분류 수집 결과 비교 기준

3유형 한국형·미국형·기타 지역형
구조 대응

91개국 글로벌 대상국 수집 흐름
테스트

Blue/Green 배포 자동화 · CI/CD · Docker · Nginx

RE:FIT

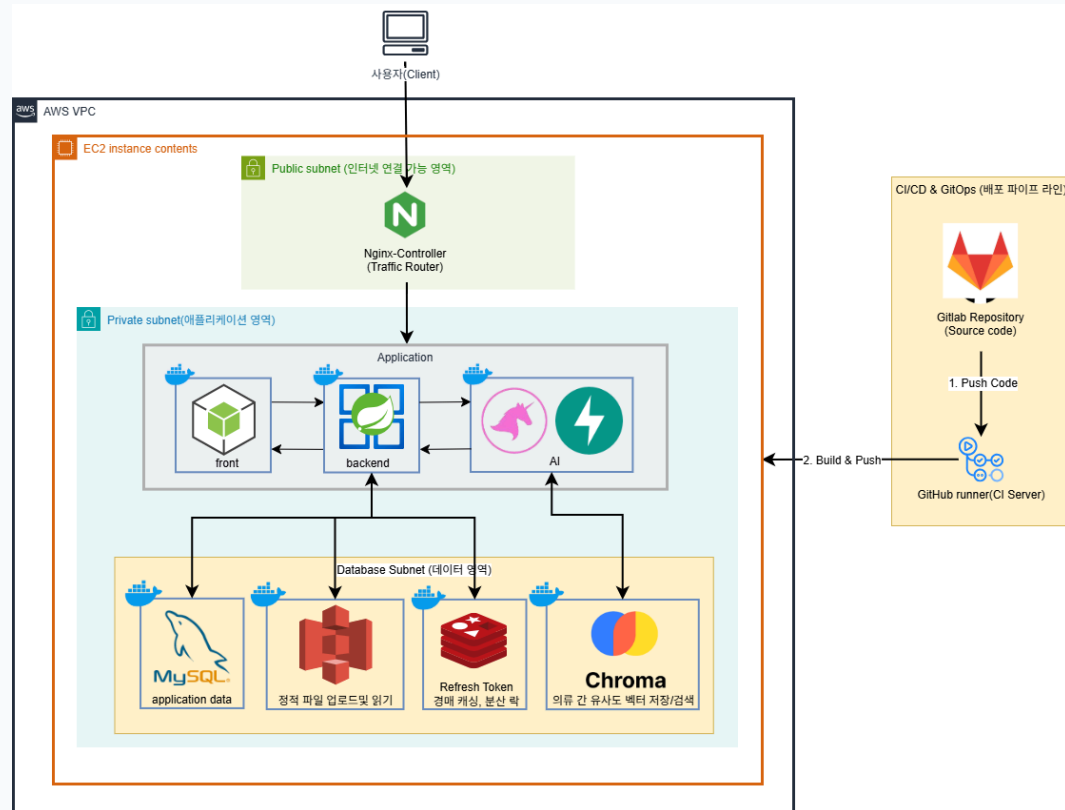
AI 가상 피팅과 중고 의류 거래 기능을 제공하는 플랫폼.

Backend·Frontend·AI 서버를 Docker로 컨테이너화하고, GitLab CI/CD와 Nginx Blue/Green 기반 배포 자동화를 구현했습니다. 총 532회 파이프라인과 205회 배포 작업을 운영하며 2월 기준 파이프라인 성공률을 87.0%까지 안정화했습니다.

📅 기간: 2026.01 ~ 2026.02 (6주)

👥 팀 구성: 6명

🔗 GitHub: github.com/dksehgud/ReFit



Nginx Blue/Green 배포 자동화 · Docker Compose 환경 분리 · GitLab CI/CD · Redisson 분산 락 · Redis 입찰 검증

Docker

Nginx

GitLab CI/CD

Redisson

Redis

MySQL

I ① 문제 정의

배포 때마다 서버가 재시작되면 실시간 경매/채팅 세션이 끊길 수 있었고, AI 서버 호출과 동시 입찰 처리에서 병목과 정합성 문제가 발생할 수 있었습니다. CI 빌드도 반복 실행되며 시간이 길어졌습니다.

I ② 문제 원인

- 단일 환경 배포라 새 버전을 띄우는 동안 기존 서비스가 영향을 받음.
- 동시 입찰과 AI 호출이 몰리면 스레드풀과 데이터 정합성 관리가 필요함.
- 매 빌드마다 의존성을 다시 내려받아 CI 시간이 길어짐.

I ③ 해결 과정

- **Nginx Blue/Green 배포 자동화**로 새 버전을 별도 컨테이너에서 실행한 뒤 설정을 갱신했습니다.
- **Redisson 분산 락**과 Redis Sorted Set으로 동시 입찰 검증 흐름을 구현했습니다.
- GitLab 의존성 캐싱, `rules:changes`, 가상 DB 테스트 환경으로 CI 흐름을 정리했습니다.

I ④ 결과 (GitLab API 실측 기준)

532회 GitLab 파이프라인 총 실행

205회 Blue/Green 배포 작업 실행

87.0% 2월 파이프라인 성공률
(안정화)

83.1초 성공 배포 소요 중앙값

대기열 제어 · Redis · 부하 테스트

Tget

예매 요청이 한 번에 몰릴 때 Redis 대기열로 입장 순서를 제어하는 공연 예매 서비스.
비활성 사용자의 토큰 만료, Redis-DB active count 동기화, 테스트 환경 기준 부하 검증을 진행했습니다.

 **기간:** 2025.11 ~ 2025.12 (4주)

 **팀 구성:** 2명 (BE 1, FE 1)

 **GitHub:** github.com/dksehgud/Tget

I ① 문제 정의

예매 오픈 시 요청이 한 번에 DB로 들어오면 커넥션이 부족해지고, 사용자가 이탈해도 active 토큰이 남아 다음 사용자의 진입이 늦어질 수 있었습니다.

I ② 문제 원인

- 예매 요청이 완충 없이 DB로 직접 유입되어 커넥션 부담이 커짐.
- 서버 재시작 시 Redis active count는 초기화되나 DB 상태는 유지 → 정합성 불일치.
- 이탈 사용자를 감지하지 못하면 active 토큰이 계속 남음.

I ③ 해결 과정

- Redis + MySQL 대기열 토큰 구조로 Waiting/Active/Expired 상태를 관리했습니다.
- Heartbeat 기반 만료로 30초 동안 응답이 없는 active 토큰을 만료 처리했습니다.
- QueueScheduler가 30초 주기로 DB active count를 Redis에 동기화하도록 구현했습니다.
- 부하 테스트로 대기열 동시성 제어, 요청 성공률, 순서 유지 흐름을 확인했습니다.

I ④ 결과 (테스트 환경 기준)

131,874회 부하 테스트 총 요청 수

100% 테스트 요청 성공률

197 RPS 테스트 최대 처리량

95.2% 대기열 순서 유지율 개선

THANK YOU

로그와 지표로 문제를 추적하고, 다시 발생하지 않는 구조로 개선하는 개발자 안도형이었습니다.

